

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE THREE

Embeddings & Vector Representations

Beginner to Industry Level • Assumes only basic Python

MODULE THREE

Embeddings and Vector Representations

Retrieval works by meaning, not by keywords — and meaning, to a computer, is a list of numbers. This module explains the single most important idea that makes modern RAG possible: the embedding. You will learn what an embedding is, how text is turned into a vector that captures its meaning, how the geometry of "embedding space" lets us measure semantic similarity, and how the three distance metrics — cosine similarity, Euclidean distance, and dot product — actually work. You will then learn to choose an embedding model wisely by trading cost against quality, and to build practical embedding pipelines with Gemini and open-source models.

WHERE WE ARE

Module 1 gave you the whole RAG loop; Module 2 turned raw files into clean chunks. Those chunks are still plain text — and a computer cannot search text by *meaning*. This module is the bridge: it covers the **embed** step of the indexing pipeline, converting every chunk into a vector. Module 4 then stores and indexes those vectors; Module 5 searches them.

3.1 Introduction

In Module 1 you saw a query about "how do I get my money back" correctly retrieve a chunk titled "Refund Policy," even though the two share almost no words. That small miracle is the entire reason RAG works, and it has a name: **semantic search** — search by meaning rather than by matching letters.

But computers do not understand meaning. They understand numbers. So before a machine can compare the *meaning* of a query with the *meaning* of a chunk, both pieces of text must first be converted into numbers — specifically, into a list of numbers called a **vector**. The technique that performs this conversion, turning a piece of text into a vector that genuinely captures its meaning, is called **embedding**.

Embeddings are the quiet foundation beneath everything in this book. The retriever in Module 5 finds chunks by comparing embeddings. The vector store in Module 4 exists solely to store and search embeddings. The advanced patterns in Module 7 — HyDE, RAG

Fusion — are clever manipulations of embeddings. If you understand embeddings deeply, the rest of RAG becomes intuitive. If they stay fuzzy, every later module rests on sand.

This module makes them concrete. We will define what an embedding is, build a strong geometric intuition for "embedding space," derive the three distance metrics with worked examples, survey the embedding models you will actually choose between, and finish by writing real embedding pipelines.

3.2 Why This Topic Matters

WHY THIS MATTERS

Two of the highest-leverage decisions in a RAG project are made in this module: *which embedding model to use* and *which distance metric to compare with*. Choose a weak embedding model and every single retrieval is slightly wrong, forever — no re-ranker or clever prompt can fully recover meaning that was never captured. Choose the wrong distance metric and your similarity scores are subtly meaningless. These are not tuning knobs you adjust later; they are foundational choices. That is exactly why interviewers ask "how do embeddings work?" and "what is the difference between cosine similarity and Euclidean distance?"

Four concrete reasons this module repays close study:

1. **Embeddings cap retrieval quality.** The retriever can only be as good as the embeddings it compares. If meaning is lost at the embedding stage, it is lost for good.
2. **The model choice has lasting consequences.** The embedding model determines vector dimensionality, multilingual ability, domain fit, cost, and latency — and you cannot change it without re-embedding the *entire* corpus.
3. **Distance metrics are constantly tested.** "Why cosine and not Euclidean?" is one of the most common technical interview questions in applied AI, and a confident answer signals real understanding.
4. **It demystifies the rest of RAG.** Once embeddings click, vector databases, similarity search, and the advanced retrieval patterns all become "obvious." This is the conceptual keystone.

3.3 A Beginner-Friendly Explanation: What Is an Embedding?

Let us build the idea slowly, with no mathematics yet.

A computer cannot read the word "dog" and feel anything about it. To a computer, "dog" is just three characters. It has no idea that "dog" is similar to "puppy", related to "cat", and utterly unlike "democracy."

An **embedding** fixes this by translating a piece of text into a list of numbers — and, crucially, it does so in a way where **the numbers carry the meaning**. Texts with similar meanings get similar lists of numbers; texts with different meanings get different lists.

Here is a deliberately tiny, made-up example. Imagine we represent every word with just **two** numbers, where the first roughly measures "is this animal-like?" and the second roughly measures "is this large?":

word	[animal-ness , size]
puppy	[0.9 , 0.2]
dog	[0.9 , 0.5]
elephant	[0.9 , 0.95]
skyscraper	[0.05 , 0.98]
democracy	[0.02 , 0.10]

Look at what these numbers do. `puppy` and `dog` have nearly identical lists — they are both very animal-like and smallish — so they sit close together. `elephant` is also very animal-like but large, so it shares the first number with `dog` but differs on the second. `skyscraper` is large like an elephant but not animal-like at all. `democracy` is neither. **Closeness in the numbers now mirrors closeness in meaning**. That is the whole idea of an embedding.

Real embeddings work exactly like this, with two differences. First, they use far more than two numbers — typically **hundreds to thousands** (Gemini's `text-embedding-004` uses 768). Each number is one *dimension* capturing some subtle aspect of meaning. Second, the numbers are not hand-assigned by humans; they are produced by a trained neural network — the **embedding model** — that learned, from enormous amounts of text, how to place meanings in the right relative positions.

DEFINITION

An **embedding** is a fixed-length list of numbers (a **vector**) that represents the meaning of a piece of text, produced by a trained neural network called an **embedding model**. The defining property of a good embedding is that **texts with similar meanings produce vectors that are close together**, and texts with different meanings produce vectors that are far apart.

The number of dimensions in the vector is called its **dimensionality**. A 768-dimensional embedding is simply a list of 768 numbers. Every chunk in your knowledge base, and every user query, becomes one such list — and that is what makes them comparable.

3.4 Real-World Analogy: A Map of Meaning

REAL-WORLD ANALOGY

Think of a world map. Every city is pinned to a location given by two numbers — latitude and longitude. Those two numbers are not arbitrary: cities that are *geographically close* have *similar coordinates*. Paris and Brussels have near-identical coordinates; Paris and Tokyo do not. By looking only at the numbers, you can tell which cities are neighbours.

An embedding model builds the same kind of map — but it is a **map of meaning**. Instead of pinning cities by geography, it pins *pieces of text* by meaning. Instead of two coordinates it uses hundreds. On this map, the sentence "how do I get a refund" is pinned right next to "our return and refund policy," while "the office is closed on Friday" sits far away in a different region entirely.

Semantic search is then nothing more than asking: "*Given the coordinates of the user's question, which pinned chunks are nearest?*"

This "map of meaning" — the space in which all embeddings live — has a proper name: the **embedding space** (or *vector space* or *semantic space*). A few properties of this space are worth fixing in your mind, because they explain everything that follows:

- **Every chunk and every query is a single point** in this space.
- **Distance in the space corresponds to difference in meaning.** Near = similar; far = different.
- **The space has the same number of dimensions as the embedding model outputs.** We cannot picture 768 dimensions, but the *logic* is identical to the 2D map — "nearest neighbours" still means exactly what it means on a map.
- **Direction can carry meaning too.** Famously, in well-trained embedding spaces, the relationship `king - man + woman` lands very close to `queen`. Meaning is encoded not just in position but in the *geometry* of the space.

Hold the map picture firmly. When we discuss distance metrics next, we are simply choosing *how to measure how far apart two points on this map are*.

3.5 Core Concepts

3.5.1 From Words to Vectors: A Short History

It helps to know what embeddings improved upon, because it explains why they are designed the way they are.

The earliest way to represent a word as numbers was **one-hot encoding**: every word in the vocabulary gets its own slot, and a word is a vector of all zeros except a single 1 in its slot. For a 50,000-word vocabulary, every word is a 50,000-long vector. This has two fatal

flaws: the vectors are enormous and almost entirely zeros (*sparse*), and — worse — *every word is equally distant from every other word*. In one-hot space, "dog" is exactly as far from "puppy" as it is from "democracy." There is **no notion of similarity at all**.

The breakthrough was the **dense embedding**: a short vector (hundreds of numbers, almost none of them zero) where similarity *is* encoded in the geometry. Early dense embeddings like Word2Vec and GloVe gave each word *one fixed* vector. Modern embedding models go one step further with **contextual embeddings** — the same word gets a *different* vector depending on the sentence around it, so "bank" in "*river bank*" and "bank" in "*savings bank*" land in different regions. The models you use in this book are contextual, and they embed whole sentences and paragraphs, not just words.

DEFINITION

A **sparse** vector is mostly zeros and very high-dimensional (e.g. one-hot encoding); it captures presence of exact words but no similarity. A **dense** vector is short and mostly non-zero; it captures meaning, so similar texts are geometrically close. Embeddings are dense vectors. (Sparse vectors return in Module 5 as the basis of keyword search and BM25 — they are not useless, just different.)

3.5.2 What "Semantic Similarity" Really Means

Semantic similarity is the degree to which two pieces of text *mean* the same thing — independent of the exact words they use. "*How do I reset my password*" and "*steps to recover account access*" are semantically very similar despite sharing no significant words. "*The cat sat on the mat*" and "*the cat did not sit on the mat*" share nearly all their words but differ in meaning.

In embedding space, semantic similarity becomes a **measurable geometric quantity**: it is how close two points are. The whole point of the embedding model is to arrange the space so that this geometric closeness genuinely tracks human-judged similarity of meaning. Measuring that closeness is the job of a **distance metric** — the subject of the next section.

3.5.3 How an Embedding Model Produces a Vector

An embedding model is a neural network — typically a *transformer*, the same family of architecture behind LLMs. You do not need to know its internals to use it, but the high-level picture is worth having:

1. The input text is broken into **tokens** (sub-word units, as in Module 2).
2. The tokens pass through many neural network layers that have been trained, on billions of sentences, to understand how words relate in context.
3. The network's internal representations are **pooled** (combined) into a single fixed-length vector — the embedding — no matter how long or short the input was.

The critical training objective is what makes embeddings useful: the model is trained so that texts humans consider similar are *pushed together* in the output space and dissimilar texts are *pushed apart*. The "knowledge" of what is similar to what is baked into the model's weights. This is why **the choice of embedding model matters so much** — a better-trained model has a more faithful map of meaning.

3.6 Distance Metrics: Measuring Similarity

Once two texts are embedded, we have two points in the embedding space and we need a number for "how close are they?" There are three metrics you must know. We will explain each with intuition, a plain formula, and a worked example.

For all examples, we will use tiny 3-dimensional vectors so the arithmetic is visible.

3.6.1 Cosine Similarity

Cosine similarity measures the **angle** between two vectors — that is, whether they point in the *same direction* — and ignores how long the vectors are.

The intuition: think of each vector as an arrow from the origin. Cosine similarity asks "do these two arrows point the same way?" If they point in exactly the same direction, the result is **1**. If they are perpendicular (unrelated), it is **0**. If they point in opposite directions, it is **-1**.

The formula:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{||A|| \times ||B||}$$

where $A \cdot B$ = the dot product (sum of element-wise products)
 $||A||$ = the length (magnitude) of A

A worked example with $A = [1, 2, 2]$ and $B = [2, 4, 4]$ — note that B points the same way as A, just twice as long:

$$\begin{aligned} A \cdot B &= (1 \times 2) + (2 \times 4) + (2 \times 4) = 2 + 8 + 8 = 18 \\ ||A|| &= \sqrt{1^2 + 2^2 + 2^2} = \sqrt{9} = 3 \\ ||B|| &= \sqrt{2^2 + 4^2 + 4^2} = \sqrt{36} = 6 \\ \text{cosine} &= 18 / (3 \times 6) = 18 / 18 = 1.0 \end{aligned}$$

The result is **1.0** — perfect similarity — because A and B point in exactly the same direction, even though B is twice as long. **This length-independence is the key property of cosine similarity, and it is why cosine is the default metric for text embeddings.** Two documents about the same topic should be judged similar even if one is a short paragraph and the other a long essay; cosine ignores that length difference and focuses purely on *direction*, i.e. *meaning*.

BEST PRACTICE

Use **cosine similarity** as your default distance metric for text retrieval. It compares meaning while ignoring document length, which is almost always what you want. The large majority of embedding models are designed and benchmarked with cosine similarity in mind, and every vector store supports it.

3.6.2 Euclidean Distance (L2 Distance)

Euclidean distance is the "straight-line" distance between two points — exactly the distance you would measure with a ruler on the map analogy.

The intuition: forget angles; just ask "how far apart are these two points?" A distance of **0** means the points are identical. Larger numbers mean further apart, hence *less* similar. Note the direction is inverted from cosine: with Euclidean distance, **smaller is more similar**.

The formula:

$$\text{euclidean_distance}(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

i.e. for each dimension, take the difference, square it, sum them all, then take the square root.

A worked example with $A = [1, 2, 2]$ and $B = [2, 4, 4]$:

```
differences: (1-2), (2-4), (2-4) = -1, -2, -2
squared:      1,      4,      4
sum:          1 + 4 + 4 = 9
distance:     sqrt(9) = 3.0
```

The Euclidean distance is **3.0**. Notice the contrast with the cosine example: cosine called these vectors *identical* (1.0) because they point the same way, but Euclidean calls them *far apart* (3.0) because one is literally twice as long as the other. This is the crucial difference — **Euclidean distance is sensitive to magnitude; cosine similarity is not**. For raw text embeddings, where magnitude often reflects length or verbosity rather than meaning, that sensitivity is usually a drawback.

3.6.3 Dot Product Similarity

The **dot product** multiplies the two vectors element-wise and sums the results. It is the *numerator* of the cosine formula, without the length normalisation.

The formula:

$$\text{dot_product}(A, B) = \sum (A_i \times B_i)$$

With $A = [1, 2, 2]$ and $B = [2, 4, 4]$, the dot product is $2 + 8 + 8 = 18$, as computed earlier.

The dot product blends *both* direction and magnitude: two vectors score high if they point the same way **and** are long. There is no fixed range — the result can be any number. Its main practical appeal is **speed**: it skips the square-root and division that cosine requires, which matters when comparing one query against millions of vectors.

Here is the key relationship that ties the three together: **if all vectors are normalised to length 1, the dot product and cosine similarity become mathematically identical**. Many embedding models (including common open-source ones) output *pre-normalised* vectors precisely for this reason — it lets a vector store use the fast dot product while getting the meaning-focused behaviour of cosine. This is not a coincidence; it is a deliberate engineering choice.

3.6.4 Comparing the Three Metrics

Metric	Measures	Range	Similar means	Magnitude-sensitive?	Typical use
Cosine similarity	Angle (direction)	-1 to 1	Value near 1	No	Default for text embeddings
Euclidean (L2)	Straight-line distance	0 to ∞	Value near 0	Yes	Some image embeddings; normalised vectors
Dot product	Direction + magnitude	$-\infty$ to ∞	Larger value	Yes	Fast search, esp. on normalised vectors

INTERVIEW INSIGHT

"What is the difference between cosine similarity and Euclidean distance, and which would you use for a RAG retriever?" — a near-guaranteed interview question. The complete answer: cosine measures the *angle* between vectors and ignores their length; Euclidean measures straight-line *distance* and is sensitive to length. For text retrieval you generally want **cosine**, because a short and a long document on the same topic should be judged similar, and cosine's length-independence delivers exactly that. Bonus point: if vectors are normalised to unit length, cosine and dot product are equivalent, so a store can use the faster dot product without changing the result.

COMMON MISTAKE

Mixing up the *direction* of the metrics. With **cosine similarity** and **dot product**, a *higher* number means *more similar*. With **Euclidean distance**, a *lower* number means *more similar*. A common bug is sorting results the wrong way — e.g. taking the *largest* Euclidean distances and returning the *least* relevant chunks. Vector stores often expose a "score" that may be a similarity or a distance depending on the metric; always confirm which direction "better" is before you sort or threshold.

3.7 Internal Architecture: How Embeddings Power Semantic Search

We can now assemble the full picture of how embeddings make retrieval work. There are two moments at which embedding happens — once per chunk during indexing, and once per query at search time — and a comparison step that connects them.

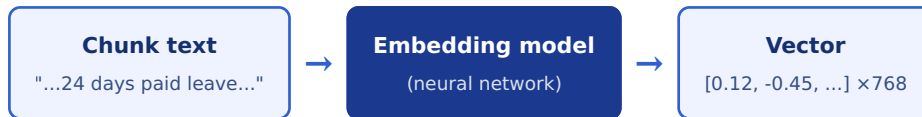
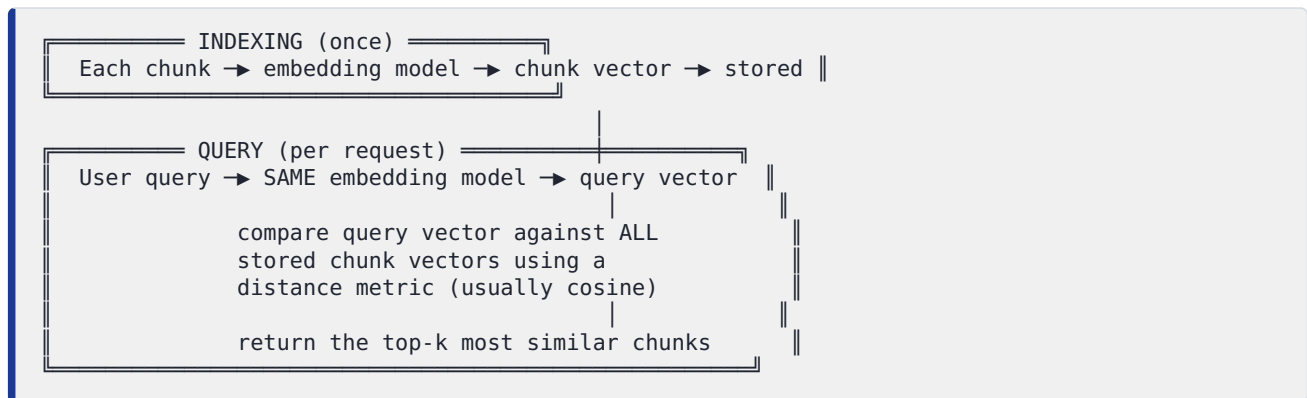


Figure 3.1 — The embedding model maps each text to a fixed-length vector. The same model is used for every chunk and for every query.

The complete data flow:



Three points in this diagram are non-negotiable, and each was foreshadowed in earlier modules:

1. **The same embedding model must be used for chunks and for the query.** (Module 1's central warning.) Two models build two different maps of meaning; a coordinate from one map is meaningless on the other.
2. **The chunk vectors are computed once and stored.** Re-embedding the corpus on every query would be absurdly slow and costly. The vectors are the durable artifact.
3. **The comparison step is "compare against all vectors."** Done naively this is slow for large corpora — which is precisely the problem the *indexing strategies* of Module 4 (IVF, HNSW) exist to solve.

3.8 Embedding Models Overview

An embedding model is a real, trained neural network, and you have many to choose from. They fall into two broad camps: **API-based** (you send text to a provider, they return vectors) and **open-source / local** (you run the model yourself).

3.8.1 Gemini Embeddings (Our Default)

Google's Gemini embedding models are accessed through the `langchain-google-genai` package and the same free-tier API key you set up in Module 1. The current general-purpose model, `models/text-embedding-004`, produces **768-dimensional** vectors and performs strongly on standard benchmarks. It is API-based: you send text, Google returns vectors. Because it shares the free tier, it costs nothing for the scale of this book.

A useful Gemini-specific feature is the **task type** hint. You can tell the API whether a piece of text is a *document to be stored* (`RETRIEVAL_DOCUMENT`) or a *search query* (`RETRIEVAL_QUERY`). The model then tailors the embedding slightly to its role, which can measurably improve retrieval. We will use this in the code section.

3.8.2 OpenAI Embeddings (For Comparison)

OpenAI's embedding models — for example `text-embedding-3-small` and the larger `text-embedding-3-large` — are also API-based and widely used. The `3-large` model produces high-quality, high-dimensional vectors; `3-small` is cheaper and lighter. They are accessed through the `langchain-openai` package and a paid OpenAI API key. We mention them because you will encounter them constantly in industry codebases and interviews, but this book uses Gemini so that every example stays free to run.

3.8.3 Open-Source Embedding Models

A large ecosystem of open-source embedding models can be downloaded and run **on your own machine or server**, with no API calls and no per-use cost. The most prominent family is **Sentence Transformers** (the `sentence-transformers` library), and you will frequently hear these specific models named:

- `all-MiniLM-L6-v2` — small (384 dimensions), very fast, light enough for a laptop. The classic starting point.
- **BGE** models (e.g. `bge-small-en`, `bge-base-en`, `bge-large-en` from BAAI) — strong quality across sizes.
- **E5** models (e.g. `e5-large-v2`) — another high-performing family.
- **GTE** and `nommic-embed-text` — further well-regarded options.

These are accessed in LangChain through the `langchain-huggingface` package. Their appeal is **zero per-call cost**, **full data privacy** (text never leaves your infrastructure), and **no rate limits**. Their cost is that *you* must provide and pay for the compute to run them, and a small local model may not match a top API model on quality.

3.8.4 The MTEB Benchmark

How do you know which embedding model is "good"? The community standard is the **MTEB** — the Massive Text Embedding Benchmark — a public leaderboard that scores

embedding models across dozens of tasks (retrieval, classification, clustering, and more) in many languages.

BEST PRACTICE

When selecting an embedding model, consult the MTEB leaderboard — but read the **retrieval** column specifically, since RAG is a retrieval task and a model can rank well overall yet only moderately at retrieval. Treat the leaderboard as a shortlist generator, not a final verdict: the best model is the one that performs best *on your own data*, which you confirm by evaluating on a sample of your real documents and questions.

3.9 Choosing the Right Embedding Model: Cost vs Quality

This is one of the most important practical decisions in a RAG project. It is fundamentally a **cost-versus-quality trade-off**, but several dimensions feed into it.

3.9.1 The Dimensions of the Decision

- **Retrieval quality.** How faithfully does the model capture meaning *for your domain*? Check MTEB retrieval scores, then validate on your own data.
- **Vector dimensionality.** More dimensions can capture more nuance but cost more storage and slightly slower search. 384 (small models) to 1536 (large API models) is the common range; 768 is a comfortable middle.
- **Cost model.** API models charge per token embedded (small but non-zero, and it adds up over millions of chunks plus every query). Open-source models have no per-call fee but require compute you provision and pay for.
- **Latency and throughput.** API models add network round-trips and are subject to rate limits. Local models' speed depends on your hardware (a GPU helps greatly).
- **Data privacy.** API models send your text to a third party — often unacceptable for sensitive or regulated data. Local models keep all text in-house.
- **Maximum input length.** Every embedding model truncates input beyond a maximum token count. Your chunk size (Module 2) must fit comfortably within it.
- **Language and domain.** Need multilingual support? Highly specialised text (legal, biomedical, code)? Some models are far stronger here than others; domain-specific models exist.
- **Operational simplicity.** An API model is one line of code and zero infrastructure. A local model means downloading weights, managing dependencies, and provisioning servers.

3.9.2 A Practical Decision Framework

If your priority is...	Lean toward...	Because
Getting started fast, low volume	API model (Gemini)	One line of code, free tier, no infrastructure
Data privacy / regulated data	Local open-source model	Text never leaves your servers
Lowest cost at very large scale	Local open-source model	No per-call fee once hardware is provisioned
Highest possible retrieval quality	Top API model, validated on your data	Generally strongest, but verify on your corpus
Specialised domain (legal, code, etc.)	Domain-specific model	Trained on text resembling yours
Multilingual content	A model with strong multilingual MTEB scores	Not all models handle non-English well

WHY THIS MATTERS

The embedding model is the hardest component to change later. Switching models means **re-embedding every chunk in the entire knowledge base** — potentially millions of API calls or hours of compute — and rebuilding the whole vector index, because vectors from the new model are incompatible with vectors from the old one. Choose deliberately at the start. A sensible rule: prototype quickly with a convenient API model (Gemini), and only commit to a different model after validating it against the privacy, cost, and quality needs of the real project.

COMMON MISTAKE

Picking the embedding model that sits at the very top of the MTEB leaderboard and assuming it must be best for you. The leaderboard reflects average performance on *benchmark* datasets, not *your* data. A model trained mostly on general web text can underperform a more modest model on, say, dense legal contracts or medical notes. The correct process is: shortlist from the benchmark, then evaluate the shortlist on a representative sample of your own documents and queries (Module 9 makes this rigorous).

3.10 Workflow: The Embedding Pipeline

Embedding sits as the third stage of the indexing pipeline. In practice the workflow is:

```

1. CONFIGURE    Choose an embedding model; instantiate it once.
   |
2. EMBED CHUNKS Pass all chunks through the model in BATCHES
   |               (embed_documents) — never one slow call per chunk.
   |
3. (STORE)      Hand the vectors to the vector store — Module 4.
   |
-- -- -- at query time -- -- --
   |
4. EMBED QUERY  Pass the user query through the SAME model
   |               (embed_query) to get the query vector.

```

Two operational points define a *good* embedding pipeline. First, **batch your document embeddings**: sending chunks in batches (which `embed_documents` does) is dramatically faster and cheaper than one API call per chunk. Second, **embed the corpus once and persist the result** — re-embedding on every run wastes time and quota; the vectors are a durable artifact saved with the vector store.

LangChain hides all model-specific detail behind one uniform interface. Every embedding class — Gemini, OpenAI, HuggingFace — exposes the same two methods:

- `embed_documents(list_of_texts)` → a list of vectors, for indexing many chunks.
- `embed_query(single_text)` → one vector, for embedding a query.

Because the interface is uniform, switching embedding providers is a one-line change.

3.11 Step-by-Step Implementation and Code

We will write three things: a direct look at what an embedding actually *is*, a hands-on demonstration of the distance metrics, and a reusable embedding-pipeline helper. Save the following as `module3_embeddings.py`.

```

"""
module3_embeddings.py
Demonstrates embeddings and similarity metrics, then provides a
reusable embedding helper. Stack: Python + LangChain + Gemini.
Install: pip install langchain-google-genai numpy python-dotenv
"""

import os
import numpy as np
from dotenv import load_dotenv
from langchain_google_genai import GoogleGenerativeAIEmbeddings

load_dotenv()
if not os.getenv("GOOGLE_API_KEY"):
    raise RuntimeError("GOOGLE_API_KEY not found. Check your .env file.")

# ----- 1. CREATE THE EMBEDDING MODEL (once) -----
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

# ----- 2. SEE WHAT AN EMBEDDING ACTUALLY IS -----
sample_vector = embeddings.embed_query("How do I get a refund?")
print(f"An embedding is a list of {len(sample_vector)} numbers.")
print(f"First 8 numbers: {sample_vector[:8]}")

# ----- 3. THE THREE DISTANCE METRICS -----
def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def euclidean_distance(a, b):
    a, b = np.array(a), np.array(b)
    return np.linalg.norm(a - b)

def dot_product(a, b):
    return float(np.dot(np.array(a), np.array(b)))

# ----- 4. DEMONSTRATE SEMANTIC SIMILARITY -----
texts = {
    "query": "How do I get my money back?",
    "relevant": "Our refund and return policy explained.",
    "unrelated": "The office cafeteria menu for this week.",
}

# Embed all three sentences in ONE batched call
labels = list(texts.keys())
vectors = embeddings.embed_documents([texts[k] for k in labels])
vec = dict(zip(labels, vectors))

print("\n--- Similarity of QUERY to each sentence ---")
for label in ["relevant", "unrelated"]:
    cos = cosine_similarity(vec["query"], vec[label])
    euc = euclidean_distance(vec["query"], vec[label])
    print(f"{label:10s} | cosine={cos:.4f} (higher=closer) "
          f"| euclidean={euc:.4f} (lower=closer)")

# ----- 5. REUSABLE EMBEDDING-PIPELINE HELPER -----
def embed_chunks(chunks):
    """Embed a list of chunk Documents; returns (texts, vectors)."""
    texts = [c.page_content for c in chunks]
    vectors = embeddings.embed_documents(texts) # batched, efficient
    print(f"[EMBED] Embedded {len(vectors)} chunks "
          f"into {len(vectors[0])}-dimensional vectors.")
    return texts, vectors

```

```
if __name__ == "__main__":
    pass # the prints above run on import; real pipelines call embed_chunks()
```

For comparison, here is the **same pipeline with an open-source local model** — note that *only the model line changes*, because LangChain's interface is uniform:

```
# pip install langchain-huggingface sentence-transformers
from langchain_huggingface import HuggingFaceEmbeddings

# Runs locally – no API key, no per-call cost, data stays on your machine
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    encode_kwargs={"normalize_embeddings": True}, # unit-length vectors
)
# embeddings.embed_query(...) and embeddings.embed_documents(...) work
# exactly as before – every downstream line of code is unchanged.
```

3.12 Line-by-Line Code Explanation

Setup. As in earlier modules, `load_dotenv()` loads the Gemini API key and the guard fails fast if it is missing.

Block 1 — create the model. `GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")` creates the embedding model object *once*. Creating it once and reusing it avoids repeated setup cost; in a real application this object is a long-lived singleton.

Block 2 — see an embedding. `embeddings.embed_query("How do I get a refund?")` sends one string to Gemini and returns its vector — a plain Python list of floats. Printing `len(sample_vector)` shows 768; printing the first eight numbers makes the abstract concrete: *this* is what "the meaning of the sentence" looks like to the machine. The numbers individually are not human-interpretable; only their *relationships* to other vectors carry meaning.

Block 3 — the metrics. Three small functions implement Section 3.6 using NumPy. `cosine_similarity` computes the dot product divided by the product of the two vector lengths (`np.linalg.norm` is the magnitude `||A||`). `euclidean_distance` is the length of the difference vector `a - b`, which is exactly the straight-line distance formula. `dot_product` is the raw element-wise sum. These mirror the formulas precisely — reading the code alongside the formulas reinforces both.

Block 4 — the demonstration. This is the heart of the file. We define three sentences: a query, a sentence that is *semantically relevant* to it, and one that is *unrelated*. The relevant sentence deliberately shares **no significant words** with the query ("money back" vs "refund," "return policy") — proving the comparison is by meaning, not keywords. All three are embedded in a single batched `embed_documents` call. We then print the cosine similarity and Euclidean distance from the query to each. You will see the *relevant* sentence score a clearly **higher cosine** and **lower Euclidean distance** than the

unrelated one. That gap *is* semantic search working — and it is the exact mechanism the retriever in Module 5 relies on.

Block 5 — the reusable helper. `embed_chunks` accepts the chunk `Document` objects produced by Module 2's `ingest.py`, extracts their text, and embeds them all in one batched call. It returns the texts and their vectors and reports the count and dimensionality. This is the literal third stage of the indexing pipeline; in Module 4 the vectors it returns get handed to a vector store.

The open-source variant. The second snippet swaps Gemini for a local Sentence Transformers model. The single most important thing to notice is that **only the model-construction line changed**. `embed_query` and `embed_documents` behave identically, so every downstream component — splitter, retriever, chain — is untouched. `normalize_embeddings=True` makes the model output unit-length vectors, which (per Section 3.6.3) makes cosine and dot product equivalent. This uniform interface is exactly why LangChain lets you treat the embedding model as a swappable choice rather than a hard-wired commitment.

BEST PRACTICE

Always embed many chunks with a single `embed_documents` call rather than looping `embed_query` over them one at a time. Batching is dramatically faster (one network round-trip instead of thousands) and cheaper, and it stays well clear of API rate limits. Looping single-item calls over a large corpus is a classic beginner performance bug.

3.13 Common Mistakes

- **Different models for indexing and querying.** Vectors from two models live in incompatible spaces; comparing them silently breaks retrieval. Use one model everywhere.
- **Embedding chunks one at a time.** Looping `embed_query` over a corpus is slow and rate-limit-prone. Batch with `embed_documents`.
- **Confusing metric direction.** Higher cosine = more similar; lower Euclidean = more similar. Sorting the wrong way returns the *worst* chunks.
- **Picking a model by leaderboard rank alone.** MTEB reflects benchmark data, not yours. Shortlist from it, then validate on your own documents.
- **Ignoring the model's max input length.** Chunks longer than the limit are silently truncated, so part of the chunk's meaning never reaches the vector.
- **Re-embedding the corpus on every run.** Embed once; persist the vectors with the vector store; reuse them.

- **Forgetting normalisation assumptions.** If a store expects unit-length vectors (for dot-product-as-cosine) and the model does not normalise, scores are off. Know whether your model normalises.
- **Treating embedding numbers as individually meaningful.** A single dimension's value means nothing on its own; only whole-vector *relationships* carry meaning.

3.14 Best Practices

- **Use cosine similarity by default** for text retrieval — it compares meaning while ignoring length.
- **Use the identical embedding model** for the corpus and for queries, always.
- **Batch document embeddings** with `embed_documents`; embed the corpus once and persist.
- **Choose the model deliberately**, weighing quality, cost, latency, privacy, max input length, and language — it is expensive to change later.
- **Shortlist with MTEB's retrieval column, then validate on your own data.**
- **Match chunk size to the model's input limit** so no chunk is silently truncated.
- **Prefer task-type hints** (e.g. Gemini's document vs query types) when the model offers them.
- **Know whether your model normalises its output**, so your metric and store agree.

3.15 Real-World Applications

- **Semantic search** over any document collection — the core of every RAG retriever.
- **Recommendation systems** — embed items and users; recommend the nearest items.
- **Duplicate and near-duplicate detection** — texts whose embeddings are extremely close are likely duplicates (useful for the de-duplication stage of Module 2's pipeline).
- **Clustering and topic discovery** — group documents by embedding proximity to reveal themes in a corpus.
- **Classification** — embed a text and classify it by which labelled examples it sits nearest.
- **Multilingual retrieval** — multilingual embedding models place a sentence and its translation near each other, enabling cross-language search.
- **Anomaly detection** — a text whose embedding is far from every cluster is an outlier worth flagging.

3.16 Mini Project: An Embedding Similarity Explorer

Make embedding space tangible by building a small explorer:

1. Create a list of about ten short sentences spanning three obvious themes — for example, some about cooking, some about space travel, some about personal finance.
2. Embed all ten in one `embed_documents` call.
3. Pick one sentence as a "query" and compute its cosine similarity to the other nine. Sort the results and print them. Confirm that sentences sharing its theme rank at the top.
4. **Keyword-vs-meaning test.** Add a sentence that is *thematically related to your query but shares no words with it*. Verify it still ranks highly. Then add a sentence that *shares several words but is about a different theme* and verify it ranks low. This proves the search is semantic, not lexical.
5. **Metric comparison.** Compute both cosine similarity and Euclidean distance for every pair. Confirm they rank the sentences consistently, and identify any pair where they disagree — then explain the disagreement in terms of vector magnitude.
6. **Extension.** Use a dimensionality-reduction tool to project the 768-dimensional vectors down to 2D and plot them. You should *see* the three themes form three visible clusters — your own picture of the "map of meaning."

This explorer turns the abstract idea of embedding space into something you have measured and seen with your own eyes.

3.17 Chapter Summary

- An **embedding** is a fixed-length vector of numbers representing the *meaning* of a piece of text, produced by a trained neural network (the **embedding model**). Similar meanings produce nearby vectors.
- Embeddings replaced **sparse** one-hot encodings (huge, zero-filled, no notion of similarity) with **dense** vectors (short, meaning-carrying). Modern models are **contextual** — a word's vector depends on its sentence.
- All embeddings live in a high-dimensional **embedding space** — a "map of meaning" where every chunk and query is a point and distance corresponds to difference in meaning.
- **Semantic similarity** is geometric closeness in that space. Measuring it requires a **distance metric**:
- **Cosine similarity** — angle between vectors; ignores length; range -1 to 1 ; higher = closer. **The default for text.**
- **Euclidean distance** — straight-line distance; sensitive to length; range 0 to ∞ ; lower = closer.

- **Dot product** — direction $\times$ magnitude; fast; equals cosine when vectors are normalised to unit length.
- The retriever uses the **same embedding model** for chunks and queries, stores chunk vectors once, and at query time embeds the query and finds the nearest chunk vectors.
- Embedding models are **API-based** (Gemini, OpenAI — easy, but per-call cost and data leaves your servers) or **open-source/local** (Sentence Transformers, BGE, E5 — free per call, private, but you provide the compute).
- Choosing a model is a **cost-versus-quality** decision across quality, dimensionality, cost, latency, privacy, input length, and language. Shortlist with the **MTEB** retrieval leaderboard, then validate on your own data. Changing models later means re-embedding the whole corpus.
- LangChain exposes every model through one uniform interface — `embed_documents` and `embed_query` — so the embedding model is a swappable choice. Always **batch** document embeddings.

3.18 Practice Exercises

1. **Conceptual.** Explain to a non-technical friend, in under 120 words, what an embedding is and why it lets a computer find a "Refund Policy" document when the user asked "how do I get my money back."
2. **By hand.** For `A = [3, 0, 4]` and `B = [6, 0, 8]`, compute the dot product, the cosine similarity, and the Euclidean distance. Explain why cosine and Euclidean give such different verdicts on the same pair.
3. **Metric choice.** For each scenario, choose cosine, Euclidean, or dot product and justify it: (a) comparing two text chunks of very different lengths; (b) searching millions of pre-normalised vectors as fast as possible; (c) comparing points where absolute magnitude is genuinely meaningful.
4. **Model selection.** A hospital wants a RAG system over confidential patient notes, with no budget for sending data to external APIs. Recommend an embedding model type and explain every reason.
5. **Code.** Run `module3_embeddings.py` and confirm the *relevant* sentence scores a higher cosine similarity than the *unrelated* one. Then add a third sentence that is the *opposite* in meaning to the query and report its score.
6. **Swap test.** Change the model from Gemini to `all-MiniLM-L6-v2`. Note the new vector dimensionality and confirm every other line of the script still runs unchanged. Explain why the swap was a one-line change.
7. **Investigation.** Embed the words "bank" in the sentences "I sat by the river bank" and "I deposited cash at the bank." Compare their similarity to the word "money." Explain what this reveals about contextual embeddings.

3.19 Interview Questions

BEGINNER

Q1. What is an embedding?

A fixed-length vector of numbers that represents the meaning of a piece of text, produced by a trained neural network. Texts with similar meanings produce vectors that are close together in the embedding space.

Q2. Why can't a RAG system just search documents with keyword matching?

Keyword matching fails when the query and the answer use different words for the same idea — "get my money back" versus "refund policy." Embeddings capture meaning, so semantically related texts are found even with no shared words.

Q3. What is the embedding space?

The high-dimensional space in which all embeddings live — a "map of meaning" where every text is a point and geometric distance corresponds to difference in meaning.

Q4. Name the three common distance metrics.

Cosine similarity (angle between vectors), Euclidean distance (straight-line distance), and dot product (direction combined with magnitude).

INTERMEDIATE

Q5. What is the difference between cosine similarity and Euclidean distance, and which is preferred for text retrieval?

Cosine measures the angle between vectors and ignores their length; Euclidean measures straight-line distance and is sensitive to length. Cosine is generally preferred for text because two documents on the same topic should count as similar even if one is much longer; cosine's length-independence delivers that, whereas Euclidean would penalise the length difference.

Q6. What is the relationship between dot product and cosine similarity?

The dot product is the numerator of the cosine formula, without the length normalisation. If both vectors are normalised to unit length, dot product and cosine similarity are mathematically identical — which is why many systems normalise vectors and then use the faster dot product.

Q7. Why must the same embedding model be used for documents and queries?

Different models produce vectors in different, incompatible spaces — different "maps of meaning." A query vector from one model compared against chunk vectors from another yields meaningless similarity scores and silently broken retrieval.

Q8. What is the difference between a sparse and a dense vector?

A sparse vector is mostly zeros and very high-dimensional (e.g. one-hot encoding); it records exact-word presence but encodes no similarity. A dense vector is short and mostly non-zero; it encodes meaning, so similar texts are geometrically close. Embeddings are dense vectors.

ADVANCED**Q9. How would you choose an embedding model for a production RAG system?**

Weigh several dimensions: retrieval quality (start from the MTEB retrieval leaderboard), vector dimensionality (storage and speed), cost model (per-token API fees versus self-hosted compute), latency and rate limits, data privacy (whether text may leave your infrastructure), maximum input length versus your chunk size, and language/domain fit. Shortlist from the benchmark, then validate the shortlist on a representative sample of your own documents and queries, because benchmark performance does not guarantee performance on your data.

Q10. What is a contextual embedding, and why does it matter?

A contextual embedding gives the same word a different vector depending on its surrounding sentence — "bank" in "river bank" versus "savings bank" lands in different regions of the space. It matters because real meaning depends on context; non-contextual embeddings (one fixed vector per word) cannot disambiguate, which weakens retrieval on naturally ambiguous text.

Q11. What is the operational cost of changing the embedding model after a system is in production?

It is high. Vectors from the new model are incompatible with the old ones, so the entire knowledge base must be re-embedded — potentially millions of API calls or hours of compute — and the whole vector index rebuilt. This is why the embedding model should be chosen deliberately at the outset.

SCENARIO-BASED**Q12. Your RAG system retrieves chunks that share keywords with the query but are about the wrong topic, while missing the chunk that actually answers it. The retrieved chunks also tend to be very long. What might be wrong, and how do you investigate?**

Two suspects. First, the distance metric: if the system is using a magnitude-sensitive metric (Euclidean or raw dot product on un-normalised vectors), long chunks can score artificially high simply because of their length — switching to cosine, which ignores length, often fixes this. Second, the embedding model may be weak for the domain, capturing surface words better than meaning — validate it on your own data and consider a stronger or domain-specific model. Also revisit chunk size from Module 2, since over-long chunks dilute meaning into vague vectors.

Q13. A team built a RAG prototype with Gemini embeddings, but the client now requires that no document text ever leaves their servers. What changes, and what is the migration cost?

The embedding model must move from an API model to a self-hosted open-source model (e.g. a Sentence Transformers or BGE model) running on the client's own infrastructure. Thanks to LangChain's uniform interface, the code change is essentially one line. The real cost is operational: the client must provision and pay for compute (ideally a GPU) to run the model, and — critically — the entire existing knowledge base must be re-embedded with the new model and the vector index rebuilt, because the old API-model vectors are incompatible with the new model's vectors.

END OF MODULE 3

Your chunks are now vectors, and you understand exactly how the geometry of embedding space turns "meaning" into something measurable. But a pile of vectors is not yet a searchable system — comparing a query against millions of vectors one by one would be far too slow. **Module 4 — Vector Stores** solves that: how vector databases store and index embeddings, how indexing strategies like IVF and HNSW make similarity search fast at scale, the create/read/update/delete operations, and a tour of ChromaDB, FAISS, Pinecone, Weaviate, Milvus, and Qdrant.